Software Requirements and Architecture

Forensic Timeline Consistency and Evidence Constraint Analyzer

Namitha Meda Boyina

FA26 - COMP-699-A - PROFESSIONAL SEMINAR

Professor: David Pitts

Rivier University

1. Problem Definition and Project Scope

I chose to develop the Forensic Timeline Consistency and Evidence Constraint Analyzer because I am interested in forensic analysis and Python software development. During an investigation, different pieces of evidence may contain different timestamps, locations, movements, and observations. Arranging these details into a timeline can be difficult, especially when some events appear to contradict each other.

For example, one record may place a person at a location at 10:00 AM, while another record places the same person at a different location at 10:05 AM. If traveling between these locations requires 30 minutes, the two events may not fit the same timeline under the given conditions. Checking such relationships manually can take time and may make it difficult to identify the exact source of a conflict.

I plan to develop a software system that allows users to enter evidence events, define relationships between events, check whether the constraints can be satisfied, and view alternative timelines when possible. The system will help users organize information and understand conflicts without changing the original evidence.

The main users will be forensic analysts, investigators, researchers, and students interested in timeline analysis. I will develop the application using Python, with a local web interface for entering events, viewing results, and exploring timelines.

The project will include event management, time-range validation, temporal and location-related constraints, conflict reporting, alternative timeline generation, and interactive visualization. I will use synthetic datasets to test known scenarios and selected National Incident-Based Reporting System (NIBRS) records for additional demonstrations where suitable fields are available.

The project will not access private investigation records, connect to live police databases, or determine guilt or innocence. It will check only the information and constraints represented in the system. Its purpose is to support timeline analysis, not to decide whether evidence is genuine or reliable.

2. Users and Stakeholders

I have identified four main stakeholder groups.

- **Forensic analysts and investigators:** They will organize evidence events, define relationships, examine conflicts, and compare possible timelines.
- **Researchers and students:** They will use the system to study constraint-solving methods and test different timeline scenarios.
- **Public data providers:** The U.S. Department of Justice provides NIBRS data that I may use for demonstration. The available fields will determine which records can be transformed into timeline events.
- **Project developer:** I will design, implement, test, and document the application and maintain its GitHub repository.

I will focus on a single-user local application for the initial version. Users will not need accounts, and the system will not support multiple users editing the same case simultaneously. I will keep original evidence information separate from calculated results to avoid accidental changes to the source data.

3. Functional Requirements

The following 40 functional requirements describe the main operations I plan to implement and test.

Evidence Event Management

- **FR-01:** The system shall allow users to create evidence events.
- **FR-02:** The system shall display all events in the current dataset.
- **FR-03:** The system shall allow users to edit existing events.
- **FR-04:** The system shall allow users to remove events after confirmation.
- **FR-05:** The system shall assign a unique identifier to each event.
- **FR-06:** The system shall allow users to enter descriptions, timestamps, locations, durations, and source references.

Time and Location Management

- **FR-07:** The system shall support exact event timestamps.
- **FR-08:** The system shall support earliest and latest possible times for uncertain events.
- **FR-09:** The system shall allow users to enter event durations when known.
- **FR-10:** The system shall allow users to record event locations.
- **FR-11:** The system shall reject time ranges in which the earliest time is later than the latest time.
- **FR-12:** The system shall allow users to define minimum travel times between locations.

Constraint Management

- **FR-13:** The system shall support constraints requiring one event to occur before another.
- **FR-14:** The system shall support constraints requiring one event to occur after another.
- **FR-15:** The system shall support minimum time gaps between events.
- **FR-16:** The system shall support maximum time gaps between events.
- **FR-17:** The system shall store references to the events involved in each constraint.
- **FR-18:** The system shall validate event references before accepting a constraint.

Consistency Analysis

- **FR-19:** The system shall check whether all supported constraints can be satisfied together.
- **FR-20:** The system shall consider uncertain time ranges during analysis.

- **FR-21:** The system shall check travel-time constraints when the required information is available.
- **FR-22:** The system shall report when no valid timeline can be found.
- **FR-23:** The system shall identify events and constraints associated with detected conflicts when possible.
- **FR-24:** The system shall explain conflicts using the relevant event details and constraint rules.

Alternative Timeline Generation

- **FR-25:** The system shall search for alternative timelines that satisfy the active constraints.
- **FR-26:** The system shall allow users to limit the number of alternatives displayed.
- **FR-27:** The system shall display the assigned times or time ranges for alternative timelines.
- **FR-28:** The system shall distinguish valid alternatives from inconsistent scenarios.
- **FR-29:** The system shall allow users to compare alternative timelines.
- **FR-30:** The system shall preserve original evidence values during alternative timeline generation.

Visualization and Results

- **FR-31:** The system shall display events on an interactive timeline.
- **FR-32:** The system shall visually identify events involved in detected conflicts.
- **FR-33:** The system shall display the details and constraints associated with a selected event.
- **FR-34:** The system shall display whether an analysis is consistent, inconsistent, or undetermined.

Data Management and Reporting

- **FR-35:** The system shall import supported CSV and JSON files.
- **FR-36:** The system shall validate imported records and report invalid data.
- **FR-37:** The system shall export event records and constraints.
- **FR-38:** The system shall export analysis results and alternative timelines.
- **FR-39:** The system shall support transforming suitable NIBRS records into the application's event format.
- **FR-40:** The system shall display a summary of analyzed events, evaluated constraints, and detected conflicts.

4. Nonfunctional Requirements

I have identified 20 nonfunctional requirements covering performance, reliability, usability, privacy, and maintainability.

Performance

- **NFR-01:** A consistency check involving up to 100 events and 300 constraints shall finish within 2 seconds in at least 95 of 100 test runs.
- **NFR-02:** A CSV or JSON file containing up to 1,000 event records shall load within 5 seconds.
- **NFR-03:** Analysis results shall appear within 3 seconds after the analysis finishes, excluding additional alternative-solution searches.
- **NFR-04:** The system shall limit the number of alternative timelines returned.

Reliability and Correctness

- **NFR-05:** The system shall return the expected result for every predefined consistency test case.
- **NFR-06:** The system shall not silently modify original evidence information during analysis.
- **NFR-07:** The system shall identify invalid or incomplete input data.
- **NFR-08:** The system shall distinguish inconsistent results from cases that cannot be fully evaluated.

Usability and Accessibility

- **NFR-09:** A new user following the instructions shall be able to complete a basic analysis within 10 minutes.
- **NFR-10:** The interface shall display clear labels and understandable conflict messages.
- **NFR-11:** The application shall support common laptop screen sizes.
- **NFR-12:** The interface shall not rely only on color to communicate conflicts.

Security and Privacy

- **NFR-13:** The initial application shall run locally without requiring evidence uploads to an external service.
- **NFR-14:** The system shall validate imported files before processing them.
- **NFR-15:** The GitHub repository shall not contain passwords, API keys, or private case records.
- **NFR-16:** The system shall distinguish original evidence information from calculated results.

Maintainability and Compatibility

- **NFR-17:** The source code shall separate event management, validation, constraint analysis, and interface logic.
- **NFR-18:** The repository shall include dependency information and setup instructions.
- **NFR-19:** The application shall run on a standard laptop using a supported Python environment.

- **NFR-20:** The main constraint-checking functions shall have automated tests.

5. System Architecture

I plan to use a modular architecture with Streamlit as the interface and Python modules for data management and analysis.

The Streamlit interface will allow users to enter events, define constraints, import datasets, and view results. The event management module will create and update event records. The validation module will check timestamps, time ranges, durations, and constraint references before analysis.

I will use NetworkX to represent relationships between events and help explore connected events. I will use a constraint-solving library to check whether the defined rules can be satisfied together. These components will have separate responsibilities because a graph representation alone cannot determine whether every timeline constraint is valid.

The analysis module will process solver results and prepare conflict details and alternative timelines. Plotly will display events and conflicts visually. Pandas and Python's JSON facilities will support data import and export.

For the initial version, I plan to use in-memory data structures and CSV or JSON files. I may use SQLite if persistent storage becomes necessary.

The main data flow will be:

User Interface → Data Validation → Event and Constraint Management → Constraint Solver → Result Analysis → Timeline Visualization

I will test the connection between these components during the proof-of-concept stage. If the solver cannot identify an exact conflicting group, the system will report the available information rather than claiming that the conflict has been fully isolated.

6. Data Design

I will organize the data into three main categories: evidence events, constraints, and analysis results.

Each evidence event will contain an event identifier, description, exact timestamp or time range, duration when known, location, source reference, and time type. The time type will indicate whether the time is exact, estimated, or unknown.

Each constraint will contain a constraint identifier, the identifiers of the related events, the constraint type, the required time gap or other parameter, a description, and a source or assumption reference when available. Constraint types will include event ordering, minimum or maximum time gaps, and minimum travel times.

Each analysis result will contain an analysis identifier, status, number of events and constraints checked, identified conflicts, and alternative timelines when available.

One event may be connected to several constraints, and each analysis may evaluate many events and constraints. I will use unique identifiers to maintain these relationships.

I will use structured Python objects during processing and CSV or JSON for importing and exporting data. If I introduce SQLite, I will create separate tables for events, constraints, and analysis results with appropriate primary and foreign keys.

I will create synthetic datasets containing known consistent and inconsistent scenarios. I will also examine public NIBRS data and transform suitable fields into the application's format. I will not assume that NIBRS contains detailed movement histories or precise timestamps for every event. Any additional constraints introduced for a demonstration will be clearly identified as assumptions.

7. Interfaces and APIs

I will use Streamlit to build the main user interface. The application will provide an event management section, a constraint management section, an analysis section, a conflict details view, an alternative timeline view, and import/export controls.

Users will be able to create or edit events, select related events, define constraints, run consistency checks, and review the results. The timeline visualization will help users compare event times and identify conflicts.

I will also define internal interfaces between the Python modules. The validation module will pass valid records to the analysis components. The solver will return a structured result containing the status and available solutions. The result analysis module will prepare the information for display.

I do not plan to develop a public REST API for the initial version because the application will run locally. CSV and JSON will provide the main external data interfaces.

8. Technology Stack and Design Decisions

I will use Python as the main programming language because it supports data processing, graph operations, and constraint solving.

I have selected the following technologies:

- **Streamlit:** For the local web interface and user interactions.
- **Pandas:** For importing, cleaning, and manipulating event data.
- **NumPy:** For numerical calculations where required.
- **NetworkX:** For representing event relationships and traversing the constraint graph.
- **python-constraint or a suitable equivalent:** For defining constraints and searching for valid assignments.
- **Plotly:** For interactive timeline and conflict visualization.
- **datetime:** For timestamp and time-range handling.
- **CSV and JSON:** For data import and export.
- **pytest:** For automated testing.
- **Git and GitHub:** For version control, source code, and project documentation.

I selected Streamlit instead of developing a separate JavaScript frontend because it will allow me to spend more time implementing and testing the main algorithm. I plan to begin with CSV and JSON rather than a database server because the initial application will use relatively small datasets.

I will test the selected constraint solver early. If it cannot support the required time ranges or alternative-solution searches, I will consider another suitable solver before continuing with the full implementation.

10. Technical Proof of Concept

I will develop a small Python program to demonstrate that the main constraint-checking approach is feasible.

The proof of concept will use synthetic events with defined time ranges and travel-time requirements. For example, one event may place a person at Location A between 10:00 AM and 10:10 AM, while another places the person at Location B between 10:05 AM and 10:15 AM. If the required travel time is 30 minutes, the program should identify that the two events cannot satisfy the stated constraints together.

I will then change the second event's time range to allow enough travel time and check whether the solver finds a valid assignment.

I will also test invalid time ranges and constraints that refer to missing events. I will record the actual output and commit the source code to GitHub.

The proof of concept will demonstrate whether the selected representation and solving method can handle the core technical problem. I will document any limitations and will not report expected outcomes as completed test results before running the program.

11. Testing and Evaluation Plan

I will evaluate the system using unit tests, integration tests, functional tests, data validation tests, and performance tests.

I will use pytest to test individual functions for timestamp validation, event management, constraint checking, and result handling. I will create synthetic datasets with known answers to verify whether the solver correctly identifies consistent and inconsistent scenarios.

I will test the complete workflow by entering events through the interface, defining constraints, running the analysis, and checking the displayed results. I will also test CSV and JSON import and export to confirm that supported information is preserved.

For performance testing, I will run 100 consistency checks using datasets containing up to 100 events and 300 constraints. The target is for at least 95 runs to finish within 2 seconds. I will also measure whether a file containing 1,000 event records loads within 5 seconds.

Where possible, I will ask volunteer users to complete the basic workflow and provide feedback on the interface.

I will save automated test results, sample datasets, timing measurements, and screenshots as evaluation evidence. I will use these results to identify completed requirements and any remaining limitations.

12. Risks and Technical Challenges

The main technical risk is that constraint solving and alternative timeline generation may become expensive when the number of events and constraints increases. I will begin with small datasets, limit the number of alternative solutions, and measure performance before expanding the system.

Another risk is incorrect conflict identification. I will test the algorithm with known contradictions and ensure that the system reports limitations when it cannot isolate an exact conflicting group. Public data may not contain detailed event times or movement information. I will inspect NIBRS fields before using them and rely on synthetic datasets for scenarios requiring precise timestamps and travel-time constraints. Library limitations may also affect implementation. I will test the selected solver during the proof of concept and consider an alternative if necessary.

Finally, the course deadline limits the time available for development. I will prioritize event management, constraint checking, conflict reporting, and basic visualization before adding optional features. I will also keep the original evidence separate from calculated results and avoid storing sensitive information in the repository.

13. Development Schedule

I will complete the project between October 14 and December 8, 2026.

- **October 14–17:** Finalize requirements, architecture, data design, and repository structure.
- **October 18–21:** Develop the technical proof of concept and test the constraint-checking approach.
- **October 22–29:** Implement event management, validation, and constraint definitions.
- **October 30–November 3:** Integrate the event manager, graph representation, solver, and result processing.
- **November 4–12:** Develop the Streamlit interface, conflict reporting, and basic timeline visualization.
- **November 13–19:** Test functional requirements, known conflict scenarios, and performance.
- **November 20–25:** Improve alternative timelines, import/export, and interface usability.
- **November 26–30:** Complete documentation, setup instructions, and supporting materials.
- **December 1–5:** Perform final testing and prepare the project demonstration.
- **December 6–8:** Review the repository, correct remaining issues, and complete the submission.

I will commit code and documentation to GitHub throughout development. I will focus first on the core algorithm and then integrate the remaining features. If a feature takes longer than expected, I will prioritize a working and tested minimum viable product.

Conclusion: I plan to develop a Python application that organizes evidence events, checks temporal and location-related constraints, identifies conflicts, and presents alternative timelines when possible. I will use synthetic datasets to verify correctness and selected public NIBRS records for additional demonstrations where suitable fields are available. I will focus on correct analysis, clear explanations, and preserving original evidence information. The completed application will help users examine whether a set of events and assumptions can fit together, but it will not determine the truth of evidence or replace professional judgment.